

C5 System — Infrastructure & Architecture Description

1. Purpose.....	1
2. Deployment topology.....	1
3. Architecture.....	2
3.1. Microservices (containers).....	2
3.2. Databases (all Oracle 19.30, HA across NOVA/TAJOV).....	5
4. Receiving flow.....	6
5. Queue pickup mechanism.....	7
6. Archiving and Peppol reporting.....	7
7. Open questions.....	8
8. Resolved decisions log.....	8
9. Scaling strategy.....	8
9.1 Throughput framing.....	8
9.2 Anticipated bottlenecks and scaling options, per component.....	9
9.3 Database-specific scaling considerations.....	10
9.4 Cross-cutting recommendations.....	11

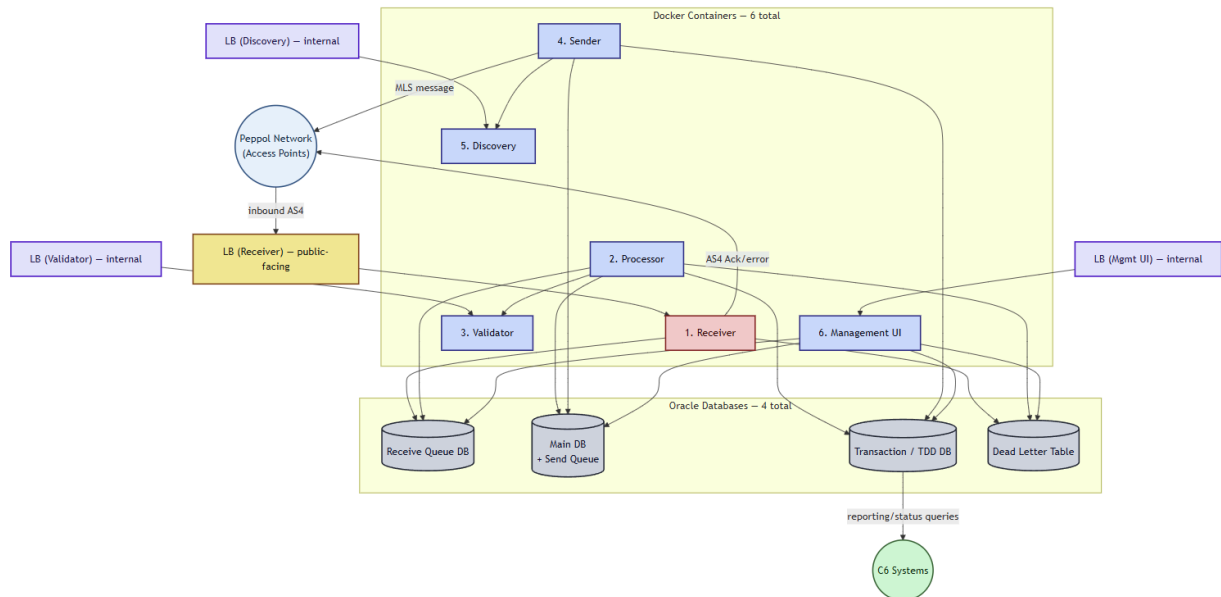
1. Purpose

C5 is a Peppol AS4 access-point system, built as a set of independently scalable, containerized microservices. The design isolates the one public internet-facing component (the Receiver) from the rest of the system, so that a breach there has a minimal blast radius.

2. Deployment topology

- **Platform:** OpenShift.
- **High availability:** deployed across **two physical sites**:
 - **NOVA** — primary
 - **TAJOV** — secondary
 - Automatic synchronization between sites.
- **Database engine: Oracle for all components.** PostgreSQL was considered for the app/main database but was dropped in favor of standardizing on Oracle everywhere, so that **cross-site replication/failover is configured and maintained using client's infrastructure standards** (Oracle 19.30 on AIX).
- **Messaging (final decision): no broker.** No Kafka, RabbitMQ, or OpenShift-provided messaging system. The "receive queue" and "send queue" are plain **database tables** that are polled/picked up by the corresponding microservices and not a separate messaging layer.

3. Architecture



https://mermaid.ai/play?utm_source=mermaid_live_editor&utm_medium=share#pako:eNp9Vltu6zYQ3QrB_CSo4IBP20JxgThCgQBJmzRuP3pVFBQ1coTl0ktRTdl4QBfRFXyIJfWWLV9_cTRn_zjw59AdmPAbs4yTjr-yZConWq1CEOVK_21yCyEGen4f4AXY7nqEfQb5y8fJ9JK6-nF8zBkWBHn_iay-lixBcXreWNp21uPPT0XkjYFq2u1d-t_vgZGKR_gfga4rsVOm_FC_TfP_-iXRlIKbtMKEvzTYh_H9j9SrM0ppK3hp1cW6ZVzDQbGwVpwbhif2-MOvlbRvebrWzw-oh-uZ1EtXZFGW0E3T2jG55Lm_uYgCmUdcPYCYvCtovCQ5HLkT_8GJTFnnXQAehBcV73K35r14gFsWCZ71osHsCfl48qhM2vO_B4Bh5dxZLx7Amlp5M3RPc7qBLs6YAOUlp-s1k-CsgyCIS5WfUYBITSiBdS1cr5Vq8evas6aM3o_soVTWKzVt4-BU5YOVRuqTAITj-12VcG10aLF-q_FrQfOCMpnYHF2hdRBMsAdA4zuQaia0hZZQ_La11jbIR9UEN2vuFLi-7NM84qWK5_rJ2Q_vx8SV0Xg0oe-aXAH6lk_cgwrRN3M89JVSH4bBTjl_nFT2BZIS71WC6Jq9XIEQXOy7lGx9dUM96WysPUprNadnNbUUzGtG6fQYuoLMmE8UBzVd_KAbxtJ_3t_fPaGt8q2uzXQ5-p6MvHafDwvUKYYOu4_TuWloFY2AHRdSrd2rQIJZFujPEkQKxV4t_9aEBY2hRBJA0u_qHalWjJM0y_ywBRhk1Cin4C_hny6UZmVEjXr6msXz2rd2bwXjGhX_mulQSe_sTLvBFnFCXzmHScpue5tnOSk7gWmN4RZxY9VNG2ccbgLQZxWg4hcJLTZObSnE9w3jYPQs_MKBOZJ0rE6tr3oijHBCmbkRMfZtw9Gm34SRwn06YNNF6dJCTMtixyRxlHDxqLYBbNnS6ylPT_Nlphz67hB8HaQdmINliRzd7lwT3MurMRZHg1UvXNb9rFa_aVo52KsGK7HtstHiG5XGMNdaLRrr_-k2LijHo76GNKtD6P3UV9xo_dTOekaO-nj0ajvuKHvo9FfVdU6jccG3og0xr4UJRh4C2JLtYg_NF_eI5bN6eUPsq2MMCS0zGelw_1RmO5r_xvm2tRS83Dxp6FZoaRypyKGIKXqSd52X0UV_416IS_T2LdtzKxbsf-A37Hv2zLHnLiFzmyylaXoGflcospG5nuOZ7tJ0Lcv5NPDfIVcyW2gwISZRcGJ688_AYimax0

3.1. Microservices (containers)

#	Module	Role	Load balancer?
1	Receiver	Public AS4 endpoint. Validates incoming AS4 messages (document type, recipient = C5 Peppol address, structural validity, valid Peppol certificate signature). Stores the SBDH + sender metadata (timestamp, certificate subject/serial, Peppol Seat ID, sender IP) in the Receive Queue DB. Returns AS4 Acknowledgment or error (EBMS:0004 NOT_SERVICED / EBMS:0101 / EBMS:0102 / EBMS:0004 OTHER).	Yes
2	Processor	Core orchestrator. Picks up a batch of items (currently an arbitrary constant, ~10 per cycle) off the Receive Queue DB via interval polling, marks them PROCESSING (own instance ID recorded for troubleshooting), calls the Validator asynchronously , stores valid TDDs + envelope + non-repudiation receipt in the Transaction DB, stores accounting metadata in the Main DB, and queues an MLS (success or error) on the Send Queue table (in Main DB).	No
3	Validator	Standalone service (ion-docval). Checks a document against OpenPeppol schematron/XSD rules, returns errors/warnings. Rules currently bundled in the node image. Called asynchronously by the Processor.	Yes
4	Sender	Picks up queued outgoing messages from the Send Queue table (in Main DB) via interval polling, looks up the recipient via Discovery, transmits via AS4. Stores the MLS + MDN/non-repudiation receipt in the Transaction DB on success. On failure, retries with exponential backoff (increasing attempts , adjusted schedule_at); after a retry limit, the message is dropped from the queue and the last error is recorded in the Main DB.	No
5	Discovery	Performs SML/SMP lookups for recipient AS4 endpoints. Used by Sender only — the Processor does not call Discovery. Not currently used by Management UI (a possible nice-to-have in a future version, at most). Kept as a separate module partly because it caches lookups, and caching is more effective with fewer discovery nodes than processor/sender nodes.	Yes

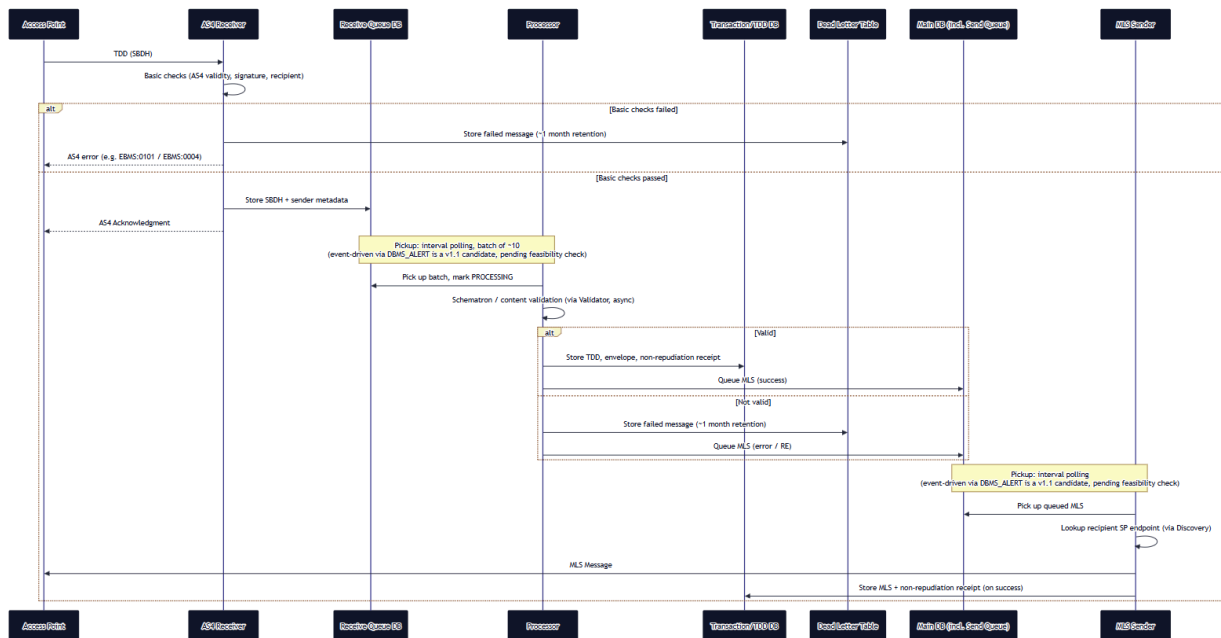
6	Management UI	Admin web interface: user/role management, receive/send queue statistics, transaction statistics, viewing received TDDs. Requires strict access control (auth + IP whitelisting) since it can read the Main DB, Transaction DB, and queues directly. Does not currently need access to Discovery or the Validator. Confirmed not need to support mobile — desktop-only usage.	Yes
---	---------------	---	-----

3.2. Databases (all Oracle 19.30, HA across NOVA/TAJOV)

Database	Contents	Notes
Receive Queue DB	Incoming SBDH (as received) + sender metadata	Isolated from Main DB for two reasons: (1) performance — keeps the Receiver's queue from becoming a bottleneck when Main DB is busy with Processor/Sender load; (2) security — the Receiver is the only public-facing service, so a breach is contained to this queue only. Could technically run on the same DB instance/config as the others if desired.
Main / App DB	Users, roles/permissions, accounting metadata, last-error records for failed sends, plus the Send Queue table	Send Queue merged in here (decision below) since Sender already needs Transaction DB access regardless, and the isolation rationale was weaker than on the receive side.
Transaction / TDD DB	Full TDD document, full envelope, non-repudiation receipt, the MLS (with its own envelope), and the MLS's non-repudiation receipt	Split out from Main DB due to very different scaling profile: up to ~100 million TDDs/year , and heavy read load from external C6 systems (checking arrival/MLS status, troubleshooting). No separate filesystem/persistent storage is used — TDD storage is DB-based, not file-based. Retention: 24 months , after which records are archived (see below). Also queried directly for Peppol reporting (handled within C5, not a separate system).
Archive Store	Records older than 24 months, copied out of the Transaction DB	Populated by a monthly scheduled archival job . Archive storage technology not yet decided. Also queried for Peppol reporting when data has aged past the 24-month window.

Dead Letter Table	Incoming messages that failed either AS4-level checks (Receiver) or content/schematron validation (Processor)	Decision: both failure types are stored here, retained for ~1 month , mainly for helpdesk/troubleshooting when an SP reports a message didn't arrive. Readable from the Management UI.
--------------------------	---	--

4. Receiving flow



https://mermaid.ai/play?utm_source=mermaid_live_editor&utm_medium=share#pako:eNg1Vm1v4kYQ_itTV1GlziE2kAPc6qQQorZScuVidB-qSNVij8kWe9e3XnNJo9xv7-zaBpw6aaWqClndnZln3p7Z5cmJZlxO4BwdPXHBdQBPx_oeMzwOjleswGMXqv1npjhbpVgck8qduhMApBBt1kqWlib7xP7sQb4oC9IKpU59hJ_PGDMOUCd8fD4cg_Pz92a6iMcTHbrl0kXiFLrOMCllyiph53VjjASeJVPpTmh6JkkrAk2gGySEv1N8TqVKoYrY2PQzZhO8GyFXfkDwZeG-6MIO9ccHXggUHgsFOuNoLI3lqGFTtsJ0Jh8o0n0mmAyjsa1YI7UR7xSmg5Hn4U6hhe9H_tQf7_Glzf7RBS4wu302jYSN4CtZbtkJj_fuCN60ryLdNcikPwfeMO5G2IF4wo8EuJIsKPZbY60Krn0rPvWi52UYEjEt81zQRc6EC0Mi4KtbSq0a0phlk3ZaN0kS8u2SzqwTW9-T60VPEHC0zSAOhL4jme5VJoJ_QPpxGJ9atMKoNBKbrAlr47ltpqVDtnpVx7r-wBG-UOX1lzQH5KLAOzYtVSeq6l8Hx2Z3yaXOWdrxbKqqEAR0thEnOqj4WIBrICLKklywYJQdZfWLUblt6isbjja7d_Q_WR06zV8KrFEmM-69BdKGueyE2z5MJ8ZoKViqjCskeJsOZ-_AjVHFI-j1IWgZgf1dmurS6TG-pW5cKsCBd6XERpH0Kk0tq4T7rsjLhyc3Md1juj1-heLE4_fgjKFICJuRfO5j_vwBpZS23GCh5Bdl_RpoCeqfSWpTzm-tEFfe4_oUqELCikKjkKfHLpkqW4DJlynGDfiF073tQogJGZjrQ71wEDvmw-ZFPqe3GnyRaU_6cQisltFYHmBSkkFPeyv-3A1uwkDz_d8OKvXnjfalWBaYDvanBXFq9E2pGpiNaWE1BUXchQs5hp9k_hXUQblb9SkusMK6lfWnykSwCkYXnjzd1xM4AFjzZIHgBNCCrqCuQypUlcu7BiOroHmcA33_tpxc4-9HBL8KexlgwBW86IVjfh7xfXV7dL4AUw2Pp9HyImYk5hU0NzyoOwIEGqylqn1PCqLK2K76JpFcQEBmVeheFCxtQGFre_Xl6F4S8ff3rN_iCzkDxliJ04WQZ2ix9Y0u-KdvaGhZzL4XO3NTciKRxGdvKyeYZ9VOjx86dWMC9

[NCGgkXUGwxITmVQEhxqjAvY155VSbBXL-FVs1uUN8uZg57RWlvslbZLNWouVVObwH-55n4FxFWl3IGt1ftlEX8Oh8rGLe6ZV6n4v9Kvsr5QU4N776Y3GKTXKd6E_S1IBT0_uqCcGGSzs2TU1FszovlpPvY7dcMsangTdWLTp1Dfhndd6_RCnq0fkmWugeO66wVj51AqxJdJ0NFf0xp6zwZtTvH_te4cwJaxpiwMtV3Dr25ZEYvw29SZo0IPc_reydIGBHQdcrcLT-inenyoZ-SQ-5dgLfG40sihM8OQ90MHnfPx_409Fw6nsD-k5c55G0BsP-eDKcjgfeyBtM33vTZ9f50_r1-pPx-fNfowfTaw](#)

Summary:

1. Access Point sends TDD (in SBDH) via AS4 to the Receiver.
2. Receiver performs basic checks; on failure, the message is stored in the Dead Letter Table (~1 month retention) and an AS4 error is returned. On success, the SBDH is stored in the Receive Queue DB and an AS4 Acknowledgment is returned.
3. Processor picks the item up, marks it PROCESSING, runs Schematron/content validation via the Validator.
4. If valid: TDD + envelope + non-repudiation receipt go to the Transaction DB, accounting metadata to the Main DB, and an MLS success message is queued.
5. If invalid: the message is stored in the Dead Letter Table instead of the Transaction DB, and an MLS error (RE) message is queued.
6. Sender picks the MLS off the Send Queue table (now in Main DB), looks up the recipient's endpoint via Discovery, transmits via AS4, and stores the MLS + MDN/receipt in the Transaction DB on success (or retries with backoff on failure).

5. Queue pickup mechanism

Processor and Sender pick up items via **interval polling** — Processor pulls a batch of items per cycle (currently an arbitrary constant, ~10) from the Receive Queue DB; Sender polls the Send Queue table (in Main DB) similarly. This is the concrete, near-term implementation.

6. Archiving and Peppol reporting

These are periodic/batch concerns rather than part of the live request-routing path, so they're intentionally left out of the architecture diagram for readability. They're still part of the architecture and are described here.

Archiving

- The Transaction/TDD DB retains data for **24 months**.
- A **monthly scheduled archival job** copies records older than 24 months out of the Transaction DB into a separate **Archive Store**.
- The archive storage technology has **not yet been decided** (see Open Questions below) — it doesn't need to be Oracle; it just needs to be queryable for the reporting use case described next.

- The Transaction DB itself is not pruned of anything within the 24-month window; only records past that age move to archive.

Peppol reporting

- Peppol reporting requirements are handled **within C5** — by querying the Transaction DB directly for recent data, and the Archive Store for data older than 24 months — rather than building or relying on a separate external reporting system.
- This was a deliberate simplification: since C6 systems already query the Transaction DB for arrival/MLS status, reusing that same data path for reporting avoids maintaining a parallel reporting pipeline.

7. Open questions

1. **Archive storage technology** — the Transaction DB archival job (monthly for records older than 24 months) needs a target archive store; the technology for this has not yet been chosen.

8. Resolved decisions log

- Failed incoming messages → **Resolved:** stored in a Dead Letter Table with ~1 month retention, covering both AS4-level and content/ schematron validation failures.
- Send Queue DB vs. Main DB → **Resolved:** Send Queue merged into Main DB. Only the Receive Queue remains a separate DB, on security/performance-isolation grounds specific to the public-facing Receiver.
- Transaction DB retention → **Resolved:** 24-month retention in the Transaction DB, with a monthly scheduled job archiving older records to a separate archive store (technology TBD — tracked as the remaining open question above).
- Peppol reporting requirements → **Resolved:** handled within C5 by querying the Transaction DB / Archive Store directly, rather than a separate external reporting system.

9. Scaling strategy

9.1 Throughput framing

At **100 million TDDs/year**, the raw message rate is actually modest:

- ~273,000 messages/day
- ~11,400 messages/hour

- **~3.2 messages/second, averaged over a full day/year**

Even assuming a bursty, business-hours-concentrated traffic pattern (say, 80% of volume in an 8-hour window, further concentrated around peak periods), sustained peak throughput is more likely in the **tens of messages/second**, not thousands. This matters for how to read the table below: the real scaling pressure over time is less "can we handle the transaction rate" and more **data volume growth, storage, and read load** (C6 queries, Peppol reporting, Management UI) against an ever-growing Transaction DB — plus the usual burst-tolerance concerns (retry storms, traffic spikes from a single large sender).

9.2 Anticipated bottlenecks and scaling options, per component

Component	Anticipated bottleneck under high load	Proposed scaling options
Receiver	CPU cost of AS4 signature/certificate validation per message; contention on Receive Queue DB inserts if many Receiver instances write concurrently	Horizontal: add more Receiver container instances behind the load balancer (stateless, scales linearly). DB: ensure Receive Queue inserts avoid hot-block contention (e.g. sequence caching, reverse-key or hash-partitioned index if using a sequential PK)
Processor	Single-threaded pickup pattern could serialize on the Receive Queue table if not implemented carefully; validation calls to Validator add latency per item; writes fan out to three destinations (Transaction DB, Main DB, Dead Letter) per item	Horizontal: add more Processor instances, each picking up items with <code>SELECT ... FOR UPDATE SKIP LOCKED</code> -style semantics so multiple instances don't block each other. Batching: process/commit in small batches rather than strictly one-by-one, to reduce round trips. Offload: keep Validator calls async/parallel per item rather than blocking the whole pickup loop
Validator	Schematron/XSD validation is CPU-intensive (XSLT-style transforms); this is the most likely per-message compute bottleneck in the whole pipeline	Horizontal: add more Validator instances behind its load balancer — it's stateless per request, so this scales well. Vertical: allocate more CPU per Validator pod, since XSLT-based validation benefits from single-thread performance as much as parallelism

Sender	A slow or unresponsive remote Access Point can tie up a Sender instance's retry/backoff cycle; Discovery lookups add latency per outbound message	Horizontal: add more Sender instances. Queue partitioning: partition/shard the send queue pickup (e.g. by recipient domain or hash) so one slow/unreachable AP doesn't starve throughput for others. Timeouts: aggressive per-attempt timeouts so backoff doesn't hold connections open indefinitely
Discovery	Cache miss rate rises with the number of distinct trading partners; SML/SMP lookups are an external network round-trip	Vertical/tuning first: increase cache size and TTL tuning before adding nodes, since the doc's original design intent was <i>fewer</i> Discovery nodes for better cache hit rate. Horizontal (limited): add nodes only if CPU/throughput — not cache effectiveness — is the constraint; consider a shared/distributed cache tier if scaling beyond a handful of nodes becomes necessary
Management UI	Not expected to be a bottleneck at this volume — low, human-driven request rate	HA only: run 2 instances behind the LB for availability, not throughput. Watch out for accidentally-heavy ad hoc queries (e.g. "view all received TDDs") against the Transaction DB — those should be paginated/limited

9.3 Database-specific scaling considerations

Database	Anticipated bottleneck	Proposed scaling options
Receive Queue DB	High insert rate combined with near-immediate delete/mark-processed churn on the same table	Keep this table small and short-lived by design (items are removed almost as soon as Processor picks them up); if contention appears, consider partitioning by insert time or moving to its own tablespace with dedicated I/O

Main DB (+ Send Queue)	Send Queue table has similar insert/pickup churn to the Receive Queue; user/permission tables are low-volume and not a concern	Same approach as Receive Queue: keep the queue table lean, consider separating the Send Queue into its own tablespace (even though it stays logically part of Main DB) if I/O contention with app-level tables appears
Transaction / TDD DB	The primary long-term scaling concern: ~100M rows/year accumulating for 24 months (~200M+ rows resident at steady state), with heavy read load from C6 systems, Management UI, and Peppol reporting queries competing with Processor/Sender writes	Partitioning: range-partition by received/sent date (e.g. monthly), which also makes the 24-month archival job a simple "drop/export oldest partition" operation instead of a row-by-row delete. Read/write separation: consider an Oracle read replica (Active Data Guard or similar) so C6/reporting/Management UI reads don't compete with Processor/Sender writes on the primary. Indexing: keep indexes aligned to actual C6/reporting query patterns (e.g. by Peppol identifier + date) rather than over-indexing, since every extra index adds write cost at this volume. Storage tiering: since this runs on Oracle 19.30/AIX already, work with the client's infra team on I/O sizing sized for sustained write throughput, not just peak
Dead Letter Table	Low volume (~1 month retention only), unlikely to be a bottleneck	No proactive scaling needed; revisit only if failure rates from a misbehaving sender spike unexpectedly
Archive Store	Mostly a storage-growth and read-latency-for-old-data concern rather than a throughput one	Once the archive technology is chosen, prioritize cheap, high-capacity storage over write throughput, since writes only happen once/month in a batch job

9.4 Cross-cutting recommendations

- **Scale services independently.** Since each of the 6 microservices is a separate container, they don't need to scale in lockstep — Validator and Receiver are the most likely to need more instances first (CPU-bound and public-traffic-bound respectively), while Discovery should scale cautiously to preserve its cache benefit.
- **Monitor before scaling.** At the throughput levels implied by 100M TDD/year, most components have real headroom; the highest-value early investment is instrumentation (queue depth, Validator latency, Transaction DB write latency) so scaling decisions are based on observed bottlenecks rather than guesses.

- **Design the Transaction DB for partition-based archival from day one** — retrofitting partitioning after the table has grown large is far more disruptive than building it in from the start, especially given the already-decided 24-month retention + monthly archival job.